

Logical Execution Time パラダイムの拡張と AUTOSAR Adaptive Platform における試作評価*

石郷岡 祐¹⁾ 羽生 浩幸¹⁾ 祖父江 恒夫¹⁾ 芹沢 一¹⁾

Extension of Logical Execution Time Paradigm and Prototype Evaluation on AUTOSAR Adaptive Platform

Tasuku Ishigooka Hiroyuki Hanyu Tsuneo Sobue Kazuyoshi Serizawa

Logical execution time (LET) paradigm has been proposed as efficient design method for multicore software development. It is also well known that LET has a disadvantage to increase latency from input to output. In this research, we propose a solution named Flexible LET, which is extension of LET paradigm. Flexible LET enables inter-LET communication based on data age required by LET-based applications to reduce latency. We developed a prototype on AUTOSAR Adaptive Platform and evaluated the performance. The evaluation results showed that our proposed method is feasible.

KEY WORDS: Information, communication, and control, AUTOSAR, multi-core CPU, Logical Execution Time (E2)

1. ま え が き

更なる安全性と快適性向上, 運行設計領域 (Operational Design Domain) の拡大に向けて, 自動運転アプリケーションの高度化・多機能化が進展しており, ソフトウェアの複雑化, 大規模化が進んでいる.

自動運転アプリケーションは, 複数の機能 (制御ロジック) で構築され, マルチコアを搭載した System-on-Chip (SoC) で実行される. マルチコアの計算資源を効率的に利用するために, レイテンシ要件と CPU 負荷要件を満たすように, 機能の実行コアを選定するコア割り当て設計が行われる. しかしながら, コア割り当て設計は開発初期時に 1 度すればよいものではなく, 開発途中の機能追加や既存機能の計算負荷の見積もり誤り, 出荷後の新規機能の追加といった様々な要因によって当初のコア割り当て設計を変更しなければならない事象が発生する. コア割り当ての再設計を実施する場合, 例えば, 負荷分散のために特定の機能のみの実行コアを変更したとしても, 他のアプリケーションの入力値を参照するタイミングや演算結果を出力するタイミングに変化を与え, アプリケーションの演算内容に影響を及ぼす (再設計の前後で結果が一致しない) 可能性がある. そのため, 再設計のたびに制御への影響をすべて検証する必要があり, 開発工数が増加する課題がある. そこで, タイミング起因の影響を局所化するための設計方法論として, Logical Execution Time (LET) パラダイムが注目されている⁽¹⁾.

*2023 年 10 月 4 日受理. 2023 年 10 月 13 日自動車技術会秋季学術講演会において発表.

1) 日立 Astemo 株式会社 (312-0062 茨城県ひたちなか市高場 2520)

LET は周期の最初に入力処理, 最後に出力処理を実施するようにタイミングを固定化し, 計算処理をそれらの間で実行する方法である. そのため, 負荷分散のために計算処理を別のコアに移動しても, 出力処理のタイミングは変化しないため, 他の処理への影響伝搬を考慮する必要がなく, 別コアに移動した計算処理が出力処理までに終了することを検証すればよい. つまり, LET によって再設計時の検証工数を削減できる.

しかしながら, 認知・計画・判断で構成される自動運転アプリケーションに LET を適用する場合, 各機能がパイプライン状に連なる構成であるために, LET 周期単位で各機能を順次実行することとなり, レイテンシが悪化する課題がある⁽²⁾.

そこで本研究では LET を拡張し, レイテンシを短縮可能とする設計方法を提案する. また, AUTOSAR Adaptive Platform (AP)⁽³⁾ 上に実装して CPU オーバヘッドとメモリ消費量, 適用工数を評価する.

2. 従 来 手 法

2.1. Logical Execution Time (LET)

制御ソフトウェアはセンサからの入力値に基づいて演算を行い, アクチュエータを制御する reactive programming モデルで実装される⁽⁴⁾. 自動車のようなリアルタイムシステム向けの reactive programming モデルは, 大きく 3 種類に分類することができる. リアルタイムプログラミングモデルの分類を図 1 に示す.

Zero Execution Time (ZET) は計算処理の時間をゼロとして考えるパラダイムであり, MATLAB/Simulink⁽⁵⁾ などで採用されている⁽⁶⁾. 一方で Bounded Execution Time (BET) は, CPU

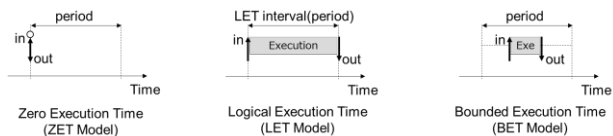


Fig.1 Classification of Real-time Programming Model

がプログラムを実行する時間まで考慮したスケジューリング理論に基づく、実行プラットフォームに依存したパラダイムである⁽⁶⁾。Logical Execution Time (LET) は、これらの中間に位置し、実行プラットフォームと非依存に設計可能とする Giotto の考えに基づいて、リアルタイムソフトウェアの実行時間を周期単位で抽象化して扱うパラダイムである⁽¹⁾。

1 章で述べたように様々な要因によって当初のコア割り当て設計を変更しなければならない事象が発生し、手法によっては検証工数が増大する課題がある。例えば、BET モデルで実装されたシステムのコア割り当てを変更する場合には、変更対象の処理と他の処理間の優先度の優劣やデータアクセスの調停等によってコア割り当ての前後で入力や出力のタイミングが変化することまで考慮して影響を分析する必要がある。つまり、システムとしての計算順序やレイテンシを再度検証する必要があり、検証工数が増加する課題がある。一方、LET モデルで実装する場合には、入力や出力のタイミングが固定化されているために、コア変更前と後でタイミング変化がない。従って、コア割り当ての変更による影響が他へ伝搬しないために検証工数を削減できる効果がある。

以上をふまえて、本研究では LET モデルを採用する。図 2 に LET で実装する例と実装しない例を示す。後者の例では、従来手法として自動車分野で広く用いられている AUTOSAR Classic Platform の implicit 通信と explicit 通信を用いる。implicit 通信ではアプリケーション実行時の最初に共有データ群から計算に用いるデータをローカルデータ群にコピー (In) する。演算処理 (Execution) ではローカルデータを用いて計算を行い、結果をローカルデータに保存する。そして最後にローカルデータの値を共有データ群にコピー (Out) する。このようにアプリケーションはローカルデータを用いることでデータの一貫性を 1 周期内で保証することが可能となる。explicit 通信では共有データ群へのアクセスタイミングの規定がなく、開発者が指定した任意のタイミングで行える。explicit 通信ではローカルデータにコピーしないために、implicit 通信と比べてデータコピー回数が少なくオーバーヘッドが小さくなる反面、共有データの値を直接参照するために、データの一貫性を保証できない課題がある。LET は implicit 通信における入力、制御処理、出力を分離した形態に近く、周期の最初で共有データ群からローカルデータにコピーを行い、周期の最後にローカルデータを共有データにコピーする。

2.2. LET 適用時の課題

図 3 に LET を適用する際の課題を示す。この例はシングル

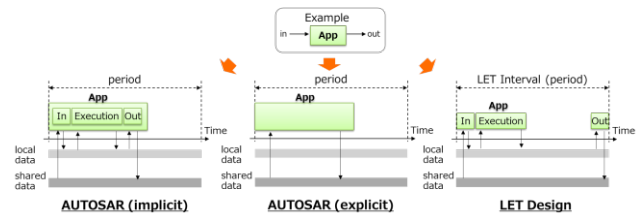


Fig.2 Data Communication of non-LET and LET

コアの CPU を複数有したマルチコア SoC であり、以降の例も同様とする。この例では 50ms 周期で実行される LET アプリケーション 1 (App1) の演算結果を、100ms 周期で実行される LET アプリケーション 2 (App2) が参照し、LET 周期の最後に出力する。App1 と App2 が並列に実行されるため、App2 が App1 の演算結果を参照できるタイミングは次の実行周期となる。例えば、App2 が App1 との入力 (in) の同時性を重視する場合には、End-to-End (E2E) レイテンシは App1 の入力時刻 T_0 から App2 の出力時刻 T_4 までの 200ms となり、App2 が最新値参照を重視する場合には、E2E レイテンシは入力時刻 T_1 から T_4 までの 150ms となる。

このように LET を適用すると、同時性重視や最新値重視に関係なく、E2E レイテンシに関わるアプリケーション数の増加に応じて、LET の周期 (LET Interval) 単位でレイテンシが増加する。従って、複数のアプリケーションが順番に実行されるパイプライン構成の自動運転アプリケーションでは、E2E レイテンシが長くなり、LET の適用が難しい課題がある。

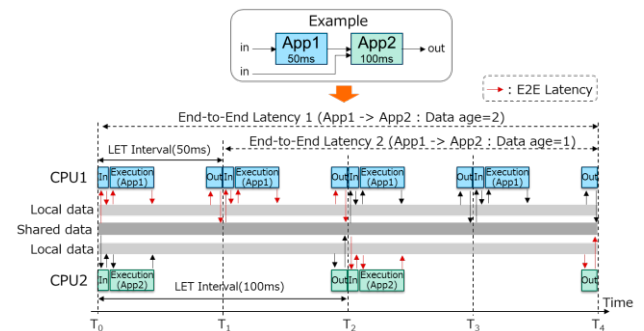


Fig.3 Issue when applying LET

3. 提案手法

3.1. Flexible LET

我々は LET パラダイムを拡張し、LET Interval が重なっている LET アプリケーション同士で直接データ通信を可能とすることでレイテンシの短縮を図る Flexible LET を提案する。Flexible LET では、LET アプリケーション同士で直接データ通信を行うための direct inter-LET communication と、Copy out されたデータを Copy in で参照する従来の LET 間通信を行うための procedural inter-LET communication を提供し、LET アプリケーションが求める data age⁽³⁾ に応じたデータ通信を可能にする。

本研究では data age をデータ通信関係があるアプリケーション間における送信(出力)と受信(参照)の相対的な実行タイミングで表す。例えば, App1 の LET interval 中に, App2 が App1 の出力データを参照する場合, data age は最新値参照を意味する 0 を用いる。App1 が前の周期で計算した結果を App2 が参照する場合, data age は前回値参照を意味する 1 を用いる。つまり, data age = n の場合, n 周期前に計算された結果であることを意味する。Data age が 0 のデータ値を参照する場合には direct inter-LET communication を, data age が 1 以上のデータ値を参照する場合には procedural inter-LET communication を用いる。本研究では, 各アプリケーションが LET interval 内に終わることとアプリケーション間通信に共有メモリ領域を用いることを前提とする。

2.2 節で述べた App1 と App2 に対して direct inter-LET communication と procedural inter-LET communication を適用したときの例を図 4 に示す。この例では App2 は同じ周期の App1 で計算された最新値(data age = 0)と, 前の周期に受信した外部入力値(data age = 1)を入力としている。App1 は共有メモリ (50ms) の data age = 0 のバッファに算出結果を保存する。App2 は App1 の演算処理(Execution)の実行後(T_{direct})に同期して起動し, data age = 0 のバッファを参照することで, 最新値を参照する。このように, 提案手法を用いると LET 周期が重なる App 間においてもデータ通信が可能となるため, 図 3 の例と比べて, E2E レイテンシを 100ms まで短縮できる。

バッファの Data age は LET interval に応じてカウントアップする。例えば, 50ms 周期で実行する App1 の出力バッファは 50ms 毎の T_1 , T_2 , T_3 のタイミングで data age をカウントアップする。data age が App の参照しない値に達したとき, バッファ値を破棄し, data age が 0 のバッファとして再利用することでメモリ消費の節約が可能となる。例えば, App1 の出力バッファは, T_0 と T_1 の区間で data age が 0 のバッファであるが, T_1 と T_2 の区間では同バッファが data age が 1 のバッファとなっている。さらに T_2 と T_3 の区間では同バッファは値がクリアされ, data age が 0 のバッファとなる。

LET interval と data age に応じてバッファを管理し, data

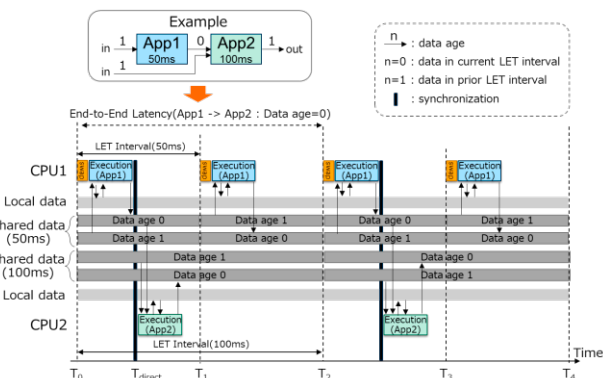


Fig. 4 Proposed Method

age に応じたバッファアクセスを可能にするソフトウェアアーキテクチャを 3.2 節で述べる。

3.2. ソフトウェアアーキテクチャ

自動運転システムでは, AUTOSAR AP の導入が進んでいる。そのため, Flexible LET が AUTOSAR AP 上で動作することが望まれる。

図 5(a)に AUTOSAR AP のソフトウェアスタックを示す。ソフトウェアスタックは, Adaptive Application(AA), AUTOSAR Runtime for Adaptive Applications (ARA)で構成される。AP は POSIX OS を実行可能なハードウェア上で動作する。図 5(b)に提案するソフトウェアプラットフォームを示す。Data age に応じてバッファを選択し, データアクセスする機能を LET library として実装する。また, LET Manager を AA として実装する。LET Manager は, 現在の実行周期を識別可能とするサイクルカウンタを管理し, サイクルカウンタに応じて data age に対応するバッファの切り替え処理を実行する。

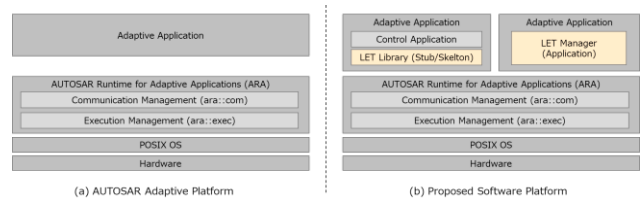


Fig. 5 Software Stack

提案するソフトウェアプラットフォームの動作シーケンスを図 6 に示す。これは, App1 と App3 を異なる CPU で実行し, App1 の後に実行する App2 と App1 の間で direct inter-LET communication を用いたときのシーケンスの例である。LET Manager は LET interval を管理するために周期的に実行する必要があるため, Deterministic Client⁽³⁾で定義する。LET Manager が実行されると, サイクルカウンタをカウントアップし, 時間経過によって変化した data age に応じて, アプリケーションが参照するバッファを切り替える処理 (swap) を行う。そして, LET Manager から ara::com::Event を発行し, 対象の AA を起動する。この例では, App1 の AA と App3 の AA に対してそれぞれ Event を発行する。そして, 最後に LET Manager は waitForActivation を用いて次の周期まで処理を終了する。図 6 は簡略化のために同一 CPU にも関わらず, LET Manager, App1, App2 が同時に実行されているような記載となっているが, 実際には 1 つの処理が順次実行される。

アプリケーション(App1, App2, App3)の AA は Event で起動されるため, Execution Client⁽³⁾で定義する。起動された AA は, 現在の周期で演算処理を実行して良いか実行条件を判定する。例えば, AA4(50ms)⇒AA5(100ms)⇒AA6(50ms)の順に計算しなければならない場合, AA5 は 50ms 周期毎に起動されるが, 100ms 周期の刻みでしか演算処理を実行しない。このようにすることで常に AA4 の後に発行する Event を AA5 に, AA6

の起動 Event の発行を AA5 に固定化できる。実行条件を満たす場合には、LET skeleton を利用して演算に必要な AA への入力値を data age に対応したバッファから取得する。その後制御演算を行い、結果を LET stub を利用してバッファに保存する。続いて実行する AA がある場合には、`ara::com::Event` を発行して AA を起動する。例えば、図 6 では App1 は App2 に対してイベントを発行している。一方で、App2 や App3 では次に実行する AA がないために、イベントを発行しない。各 AA は最後に次の周期でイベント受信時に再度起動できるように、イベント待ち(`waitForActivation`)で処理を終了する。

異なる CPU で AA を同期して順次実行する場合、AUTOSAR Classic Platform のスケジューリングテーブルのように周期にオフセットを加える実装方式も考えられる。しかしながら、AUTOSAR AP ではオフセット起動には対応していないため、前述のようなイベントを用いた方法が有力であると考えられる。

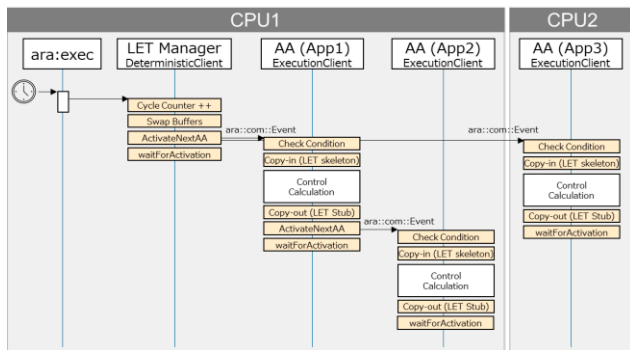


Fig. 6 Sequence Diagram

3. 3. LET Manager と LET Library

LET Manager はサイクルカウンタと、各 AA が求める data age に基づいて、各 AA が参照するバッファアドレスを切り替える Swap 処理を実行する。これにより各 AA は LET Stub と Skeleton 経由で、期待する Data Age のバッファにアクセスできる。データ保存時には Stub を、データ参照時には Skeleton を用いる。サイクルカウンタや各データバッファは共有メモリ領域に置かれ、LET Manager や LET Library からアクセスされる。

マルチバッファ構成と Swap タイミングの例を図 7 に示す。(a)の Receiver-Centric Buffer Swap 方式では、参照したい各アプリケーションが必要な data age に基づいてバッファを用意し、参照アプリケーションの周期に応じてバッファを切り替える方式である。更新アプリケーションと参照アプリケーションの間の周期差が大きいほど、切り替えるオーバーヘッドが小さくなる利点がある。一方で参照アプリケーションの数が多いほど、バッファ数とコピー回数が多くなり、更新オーバーヘッドが増える特徴がある。(b)の Sender-Centric Buffer Swap 方式では、参照アプリケーションが共有するバッファを設け、更新アプリケーションの周期に応じてバッファを切り替える方式である。参照アプリケーションの数が多い

ても、更新オーバーヘッドを抑えられる利点がある。一方で、更新アプリケーションと参照アプリケーションの間の周期差が多いときに、参照アプリケーションがアクセスしないにも関わらず、バッファを切り替えるために、無駄な切り替えが頻発してしまう特徴がある。我々は、バッファ切り替えに要する時間よりもデータをバッファに保存するコピー処理に要する時間の方が長いことを考慮し、基本的には Sender-Centric Buffer Swap 方式を採用するが、アプリケーションの特徴に応じて Receiver-Centric Buffer Swap 方式も併用する方針とした。図 4 の例では、AA 間の周期差が小さいために Sender-Centric Buffer Swap 方式を用いている。

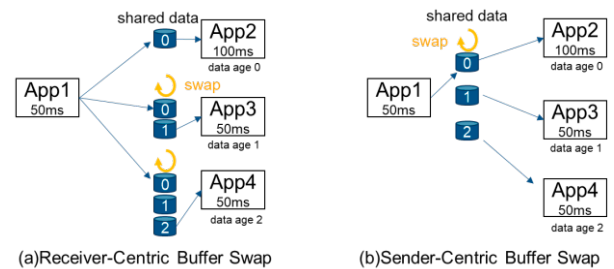


Fig. 7 Multiple Buffer Structure and Swap Timing

LET Stub と Skeleton は AA 間で通信されるデータごとに用意する必要があるため、データ数が増加すると設計工数が増大する懸念がある。そこで解決策として、LET Configuration Code Generator を開発する。Code Generator に AA 間の要求 data age 付きのデータフロー、各 AA の実行条件、実行コア、実行優先度の情報を Direct Acyclic Graph (DAG) 形式で入力することで、LET Manager のコンフィギュレーションと LET Library のソースコードを生成する。バッファ数に関しては、receiver-centric buffer swap 方式の場合は各 AA の要求 data age に 1 を加えた数のバッファが用意され、sender-centric buffer swap 方式の場合は、参照する AA 内で最も大きい data age に 1 を加えた数のバッファが用意される。

4. 方式評価

4. 1. 評価条件

提案手法によるレイテンシ削減量を評価するため、自動運転関連アプリケーションの LiDAR (Light Detection And Ranging) アプリケーションを例題に用いる。LiDAR アプリケーションは図 8 に示すように 15 個の処理で構成され、3 次元点群や占有グリッドマップを作成し、フリースペースを検出する。一連の処理を 100ms 周期内で実施する。つまり、要求レイテンシは 100ms となる。本評価では各処理(P)を一つの AA と見なし、4 コアに割り当てたときのレイテンシを評価する。各コアの性能は同じとし、各処理に示された数字は処理を実行するために必要な 1 コアでの実行時間(ms)を意味する。

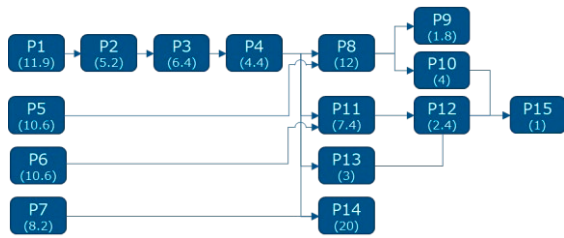


Fig. 8 Example of LiDAR Application

4.2. 評価結果：レイテンシ

評価結果を図9に示す。LET⁽¹⁾を適用すると、Pの実行周期がLET Intervalとなるため、P1-P2-P3-P4-P8-P10-P15あるいはP1-P2-P3-P4-P11-P12-P15の実行パスのレイテンシが最長の701msとなる。レイテンシ短縮のために、Pの周期を短くする方法が考えられるが、CPU負荷が増大し、現実的でない。そこでPの周期を短くしつつ、必要な周期タイミングでのみPを実行する sub-scheduling 方式⁽⁷⁾を比較対象に選択した。LiDARアプリケーションで最も処理時間が長いPはP14の20msとなるため、20msでLET intervalを定義することとする。その結果を図9(a)に示す。レイテンシが141msとなり、目標値を満たすことができない。そこで、P2, P3, P4などの処理時間の後が空き時間が多く、並列処理がない部分に対して、Pを統合して一つのAAとするグルーピング方式⁽⁸⁾を適用したときの結果を図9(b)に示す。レイテンシが76msとなり、要求値を達成することが可能となる。しかしながら、AA統合のための工数増加や、今後新たなアプリケーションが追加された場合の負荷分散の阻害要因となる可能性がある。LiDARアプリケーションのクリティカルパスは図8によるとP1-P2-P3-P4-P14であるが、P14の入力がP4とP7であるために、P14までグルーピング対象とすることができない。従って、22.1msのCPU空き時間が生じる。同様にP15の入力がP10とP12であるためにグルーピング対象とすることが困難であり、7.2msの空き時間が生じる。つまり、従来手法⁽⁷⁾⁽⁸⁾を組み合わせても、CPUの資源効率が悪化するケースがあることがわかる。

提案手法を適用したときの結果を図9(c)に示す。LET区間に関わらずdirect inter-LET communicationが可能となるために、P4の後にイベントを発行し、P8, P11, P13, P14を起

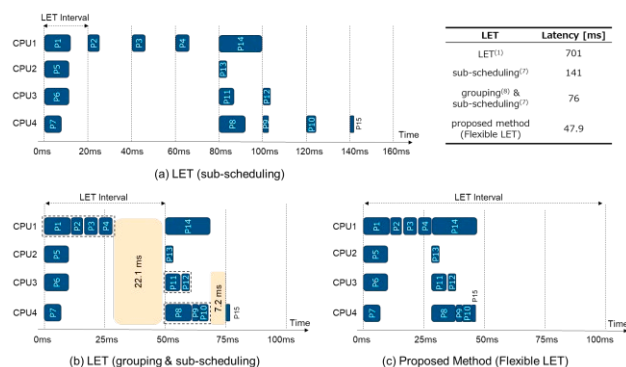


Fig. 9 Latency Results

動でき、CPU 空き時間の発生を防ぐことができる。Swap 時間をゼロとしてレイテンシを算出した結果、レイテンシは47.9msとなり、従来手法(図9(b))と比べて28.1msのレイテンシ削減を確認できた。

5. 実装評価

5.1. 評価環境

提案手法の実装評価を行うために、3.3節で提案したLET ManagerとLibraryのプロトタイプを開発し、Renesas製R-Car S4(4 Cortex A55(Dual Core 1.2GHz))⁽⁹⁾に実装した。POSIX OSにはBlackBerry製のQNX、AUTOSAR APにはバージョンがR20-11のKPIT製を用いた。本評価では、CPUオーバーヘッド、メモリ消費量、適用工数に関して評価する。

測定箇所を図10に示す。提案手法はLET Managerとlibraryを搭載したAAで構成される。LET Managerは周期に応じてサイクルカウンタを更新し、data ageに応じてAAがアクセスするバッファを切り替える処理(①)と、AAをEventで起動する処理(②)からなる。AAはSOME/IPのEventで起動されるまでの待ち時間が生じた後(③)、実行条件を判定する処理(④)、AAが求めるdata ageのバッファを参照する処理(⑤)、制御処理を演算する処理(⑥)、AAの演算結果を共有メモリに保存する処理(⑦)、後に実行すべきAAを起動する処理(⑧)、次の周期にEventで起動されるための準備処理(⑨)の順に実行される。

⑤⑦はAUTOSARのimplicit通信と同様にAAの計算に必要なデータの入力と出力処理を行う。Flexible LETで必要なdata age毎のバッファ切り替えはLET Managerで行われるために、AAに対しての入力と出力処理はAUTOSARで差異がないと考える。同様にFlexible LETにおける④⑧⑨もAUTOSAR APのライブラリを利用しているのみであり、AUTOSAR AP単独利用時との差異はないため、測定対象から除外することとする。

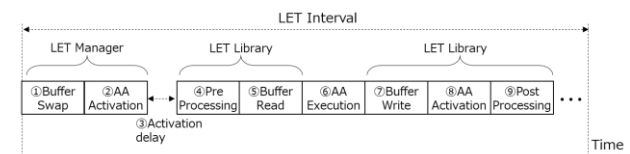


Fig. 10 Measurement Points

5.2. 評価結果：CPUオーバーヘッド

本評価ではLET intervalを50サイクルの間実行し、LET ManagerとAAの実行時間、タイミングを測定した。

図11(a)にLET Managerの実行時間を示す。LET Managerから起動するAAを最小の1個の場合とR-Car S4のコア数と等しい8個を実行する場合を比較すると、起動数が多いほど実行時間が長くなることがわかる。

図11(b)にLET Managerの処理が終了してからAAがEventで起動するまでに要する時間を示す。Eventが発行された順番にAUTOSAR APとOSが処理を行っているため、発行順に起動時間が長くなっていることがわかる。

図 11(c)に 2AA 間のデータの 1 個, data age を 1 に固定し, バッファサイズを 100 バイト, 1000 バイト, 10000 バイトに変えたときの実行時間を示す. LET Manager (①+②), バッファの保存(⑦), バッファの参照(⑤)においては, バッファサイズの変化が実行時間に与える影響が小さいことがわかる.

図 11(d)にデータサイズを 1000 バイト, data age を 1 に固定し, AA 間で通信するデータの数を 1 個, 10 個, 30 個に変えたときの実行時間を示す. データ数の増加に応じて各々の実行時間が増加していることがわかる.

以上の測定結果によると, 例えば, 8 コアで各 AA を起動し, データ通信で用いるバッファ数が 30 個であるときの LET Manager のオーバーヘッドは約 1.7ms(図 11(d)の Manager (①+②)の 30), AA が起動されるまでの時間が約 5.2ms(図 11(b)の 8 個目の AA)となる. 4 章で用いた自動運転関連アプリケーションのように実行周期が 100ms の場合, 提案手法適用時の増加オーバーヘッドが 7%未満となり, 許容可能であると考えられる.

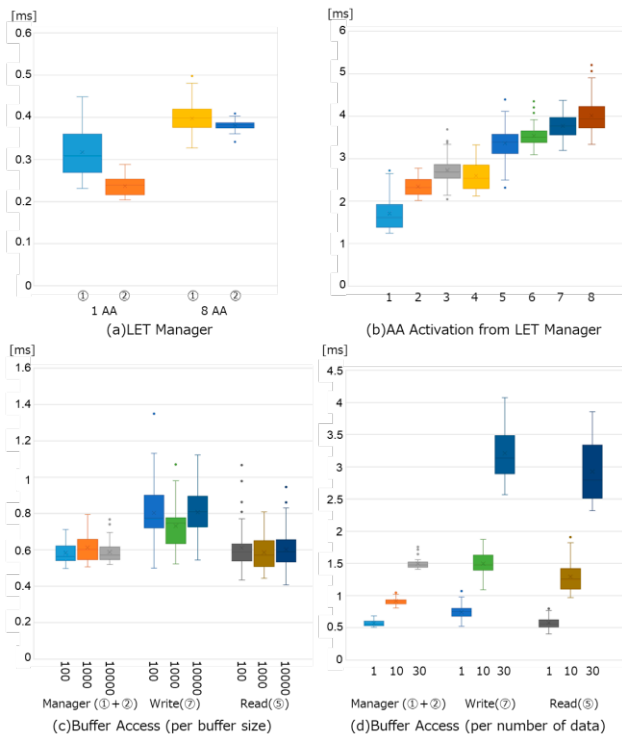


Fig.11 Measurement Results (Execution Time)

5.3. 評価結果：メモリ消費量

図 12(a)に 1000 バイトのデータ数に応じた LET Manager のメモリ消費量を示す. データ数に応じてバイナリサイズと stack と共有メモリ領域が増加していることがわかる. 図 12(b)に AA 数に応じた LET Manager のメモリ消費量を示す. AA 数に応じてバイナリが増加していることがわかる.

測定結果の傾向に基づいてデータ数が 100 個, AA が 20 個のときのメモリ消費量を推定(外挿)すると, LET manager は約 2.6Mbyte となる. これは R-Car S4 のフラッシュメモリ(eMMC)のサイズ(32GB)の約 0.01%にあたり, 許容可能であると考えられる.

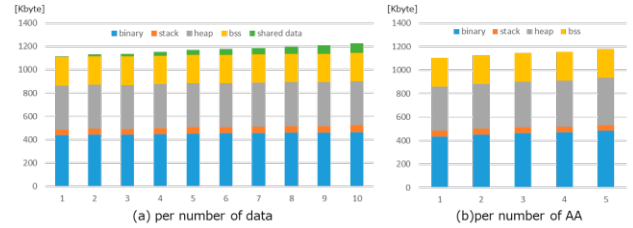


Fig.12 Measurement Results (Memory Consumption)

5.4. 評価結果：適用工数

提案手法適用時の改善効果を評価する. 評価で用いた例において AUTOSAR AP では 1 つの AA に対して 229 項目を設定する必要があったが, 提案手法では Code Generator の入力となる 16 項目 (AUTOSAR AP と共通の 15 項目 + 新規項目の data age) の設定でコードを生成でき, 設定項目を 93%削減できた.

5. まとめ

本研究では LET 適用時のレイテンシ短縮を可能とする Flexible LET を提案し, R-Car S4 と AUTOSAR AP を活用したプロトタイプ評価を通じて, 実現可能な見込みを得た. 今後の予定として, LET Manager の実装最適化を計画している.

参考文献

- (1) Christoph M. Kirsch, Ana Sokolova: The logical execution time paradigm, *Advances in Real-Time Systems*. Springer, pp.103-120 (2012)
- (2) Arne Hamann, Dakshina Dasari, Simon Kramer, Michael Pressler, Falk Wurst: Communication Centric Design in Complex Automotive Embedded Systems, *29th Euromicro Conference on Real-Time Systems*, pp.10:1-10:20 (2017)
- (3) AUTOSAR: AUTOSAR Adaptive Platform, <https://www.autosar.org/standards/adaptive-platform> (参照 2023.10.03)
- (4) Nicolas Halbwachs: Synchronous programming of reactive systems - A tutorial and commented bibliography, *Lecture Notes in Computer Science*, pp.1-16 (1998)
- (5) The MathWorks Inc.: Simulink (online), <https://www.mathworks.com/products/simulink/> (参照 2023.10.03)
- (6) Thomas A. Henzinger, Benjamin Horowitz, Christoph Meyer Kirsch: Embedded Control Systems Development with Giotto. *ACM SIGPLAN Workshop on Languages, Compilers, and Tools for Embedded Systems*, pp.64-72 (2001)
- (7) Masataka Ogawa, Shinya Honda, Hiroaki Takada: Efficient Approach to Ensure Temporal Determinism in Automotive Control Systems, *International Symposium on Embedded Computing and System Design* (2018)
- (8) Matthias Beckert, Mischa Möstl, Rolf Ernst: Zero-Time

Communication for Automotive Multi-Core Systems under SPP Scheduling, IEEE International Conference on Emerging Technologies and Factory Automation (2016)

(9) Renesas: R-Car S4,
<https://www.renesas.com/jp/ja/products/automotive-products/automotive-system-chips-socs/r-car-s4-automotive-system-chip-soc-car-servercommunication-gateway> (参照
2023. 10. 03)